



POLITECNICO
MILANO 1863

Intelligent Job Management – Jan Wangrat

Intelligent Job Management

Final Report

Author: Jan Wangrat

Supervisor: Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini

Date: May 13, 2026

Deliverable:	Final Report
Title:	Intelligent Job Management
Author:	Jan Wangrat
Supervisor:	Prof. Danilo Ardagna, Ph.D. Candidate Federica Filippini
Version:	1.0
Date:	May 13, 2026

Contents

Table of Contents	3
List of Figures	4
List of Tables	4
1 Introduction	5
2 System Architecture	6
2.1 Deployment topology	6
2.2 Scheduling pipeline	8
2.3 State machine	12
2.4 Concurrency invariants	12
2.5 Cross-node checkpoints and image variants	13
3 Configuration & Job Submission	14
3.1 Cluster Configuration	14
3.1.1 Node Configuration (nodes_config.json)	14
3.1.2 GPU Energy Costs (gpu_energy_costs.json)	15
3.2 Job Submission	15
3.2.1 Request Body	15
3.2.2 Response (201 Created)	16
3.3 Full API Overview	16
3.4 Runtime environment	17
4 End-to-End Scenarios	18
4.1 Objective and cost model	18
4.2 Two-GPU placement	18
4.3 Scenario 1 — single type	19
4.4 Scenario 2 — two types	20
4.5 Reproducibility	21
References	22

List of Figures

1	Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount.	7
2	Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker.	9
3	Profiling cycle.	10
4	Optimizer cycle.	11
5	Standard run execution. The worker performs an atomic conditional UPDATE . . . WHERE status = ANY(RUNNABLE_STATUSES) so concurrent stops / duplicate dispatches cannot race a row into RUNNING. On exit the worker fires both <code>ijm_slot_freed</code> (which releases the API’s NodeSlots permit) and <code>ijm_schedule</code> (which wakes the scheduler) in the same transaction.	12
6	Scenario 1 timeline (v9, 2026-05-12). Taller bars in the profile phase are the 2-GPU profile cells. Slot count ≤ 2 on each node at every instant.	19
7	Scenario 2 timeline (v14, 2026-05-13). Taller bars are the 2-GPU profile cells. URG-S preempts JOB3 at $t=8.95$ min; URG-B preempts JOB4 at $t=10.03$; user-stop on JOB4 at $t=10.33$; JOB7 queued at $t=10.38$ (dotted) and dispatched once URG-S vacates A40 at $t=13.78$	20

List of Tables

1	Node configuration fields.	14
2	Job creation request fields.	15
3	API endpoints.	16
4	Key environment variables.	17
5	Bundle hourly rates and per-priority tardiness weights. Two-GPU bundles include a small co-location discount.	18
6	Measured per-epoch wall-clock and GPUcost for <code>cnm_big</code> ; both 2-GPU bundles beat their 1-GPU sibling.	18
7	Scenario 1 submissions.	19
8	Per-bundle cost the optimiser sees at the S1 round; slack deadlines so TardCost= 0. . . .	19
9	Optimiser plan comparison for URGENT at the S2 round.	19
10	Scenario 2 submissions.	20
11	Per-bundle profile data the optimiser sees at the S1 round.	20
12	Optimiser plan comparison for URG-S (S2) and URG-B (S3).	21

1 Introduction

2 System Architecture

2.1 Deployment topology

IJM runs as a small distributed system. The FastAPI *API* (which also hosts the *ProfilingScheduler*, *JobDispatcher*, *NodeSlots* semaphores and the optimizer client) runs on the operator's machine. A lightweight FastAPI *worker* runs on every GPU node and owns the local Docker daemon. PostgreSQL runs on one of the GPU nodes. Communication is plain HTTP over SSH tunnels, so the API reaches each worker at a stable `http://localhost:800x` alias regardless of where it actually sits. An optional GPUspb *optimizer* HTTP service is reached from the API.

Current Prototype Architecture (distributed)

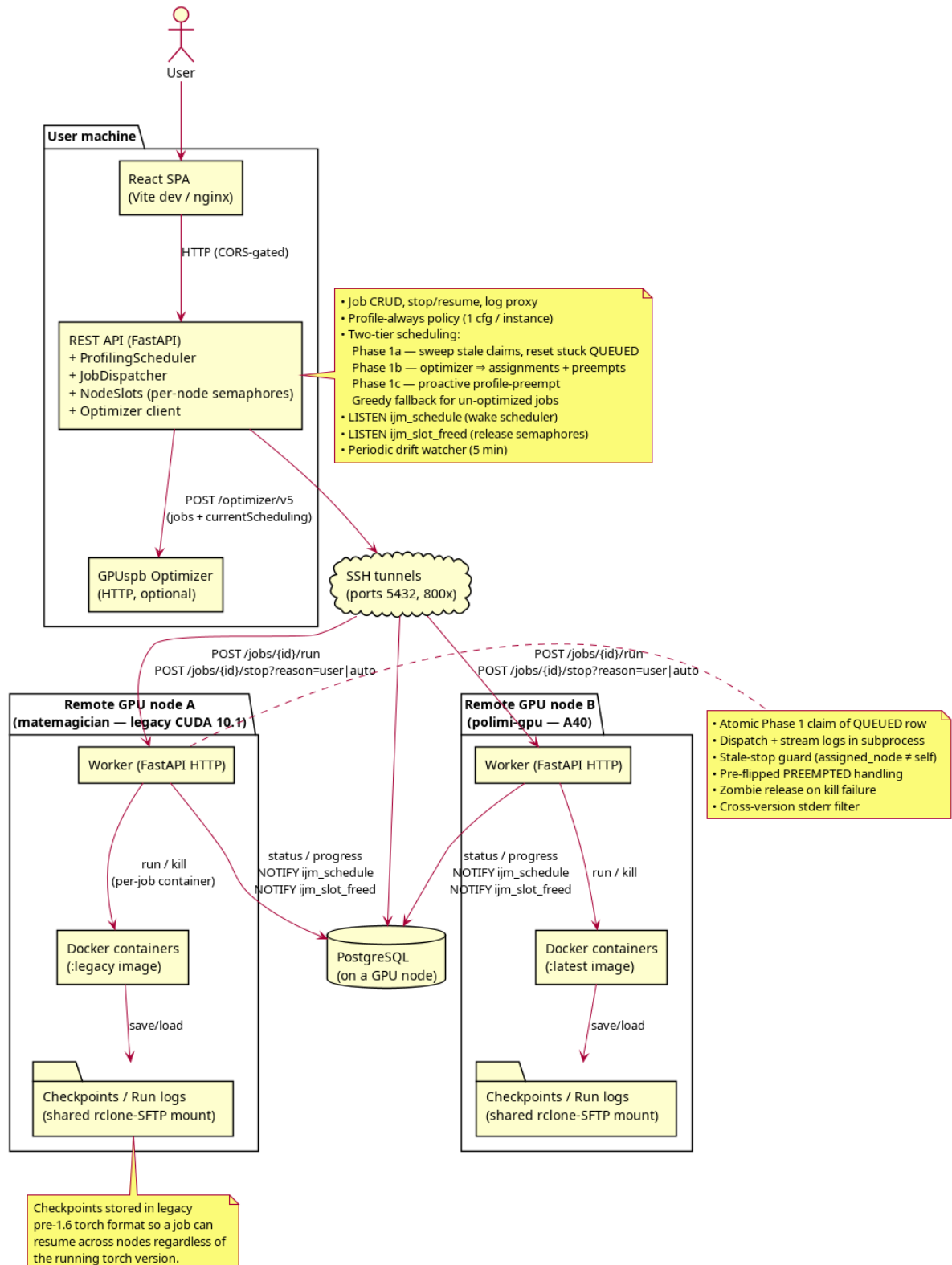


Figure 1: Distributed prototype: SPA + API + optimizer on the operator’s machine, workers + Docker on each GPU node, PostgreSQL on a GPU node, all glued together over SSH-tunneled HTTP. Checkpoints written by one node are visible from the other via a shared rclone-over-SFTP mount.

Single-process docker compose on the operator’s machine is supported for development but is not

the deployment target: any tradeoff between local-dev simplicity and distributed correctness resolves toward the distributed topology.

2.2 Scheduling pipeline

A single scheduling pass — `_schedule_waiting_jobs` in `app.py` — handles every dispatch decision. It is invoked whenever a job is submitted, a profiling run finishes, a slot is freed, or every 60 s by a watchdog. Notices are coalesced through a single `asyncio.Event` and rate-limited to at most one pass every five seconds so the cluster has time to apply the previous plan before the next one is computed.

The pass walks four phases under a single `schedule_lock`:

Phase 0 Sweep stale profile claims (rows in `profiling_results` with `duration_seconds` IS NULL whose owning instance no longer exists or was repurposed) and reset stuck QUEUED rows whose `assigned_node` was set ≥ 30 s ago without the worker advancing them.

Phase 1b Call the optimizer (outside the lock for its 30 s HTTP timeout) and apply its assignments via conditional UPDATE ...WHERE `assigned_node` IS NULL; collect its preempt list.

Greedy fallback For QUEUED rows the optimizer did not place (typically rows still owing a profile run), the `ProfilingScheduler` picks one profile config or a profiled standard slot.

Phase 1c If a row still has no slot and still owes a profile, find the cheapest set of low-priority RUNNING jobs whose eviction would free enough GPUs for the next unprofiled config (the *profile-always* policy).

Outside the lock the pass spawns one task per preempt and one per dispatch. Each dispatch goes through `JobDispatcher.dispatch_with_slot`, which acquires the destination's `NodeSlots` permit before sending POST `/jobs/{id}/run` to the right worker. Permits are released when the worker emits a NOTIFY `ijm_slot_freed` carrying the freed (`node_id`, `n_gpus`) pair.

Figure 2 shows the submission path, Figure 3 the cross-instance profile cycle, Figure 4 an optimizer-driven preempt+migrate round, and Figure 5 the standard run.

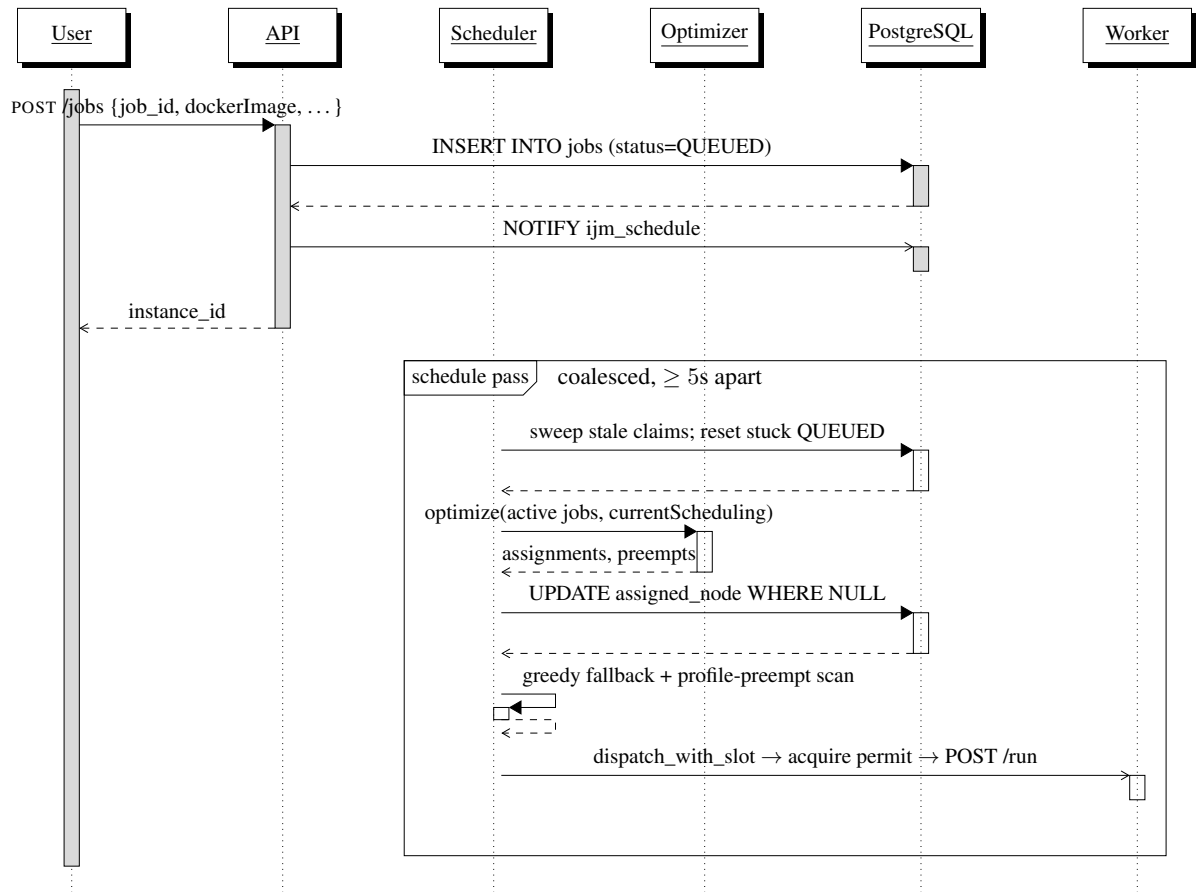


Figure 2: Submission and scheduling. The API only writes the row and notifies the scheduler; placement is decided by the next scheduling pass. The optimizer call is made outside the schedule lock so its 30 s HTTP timeout cannot block other passes; stale optimizer suggestions are safely filtered by Phase 1b’s conditional update. Slot semaphores ensure that no more containers than GPUs are dispatched to any worker.

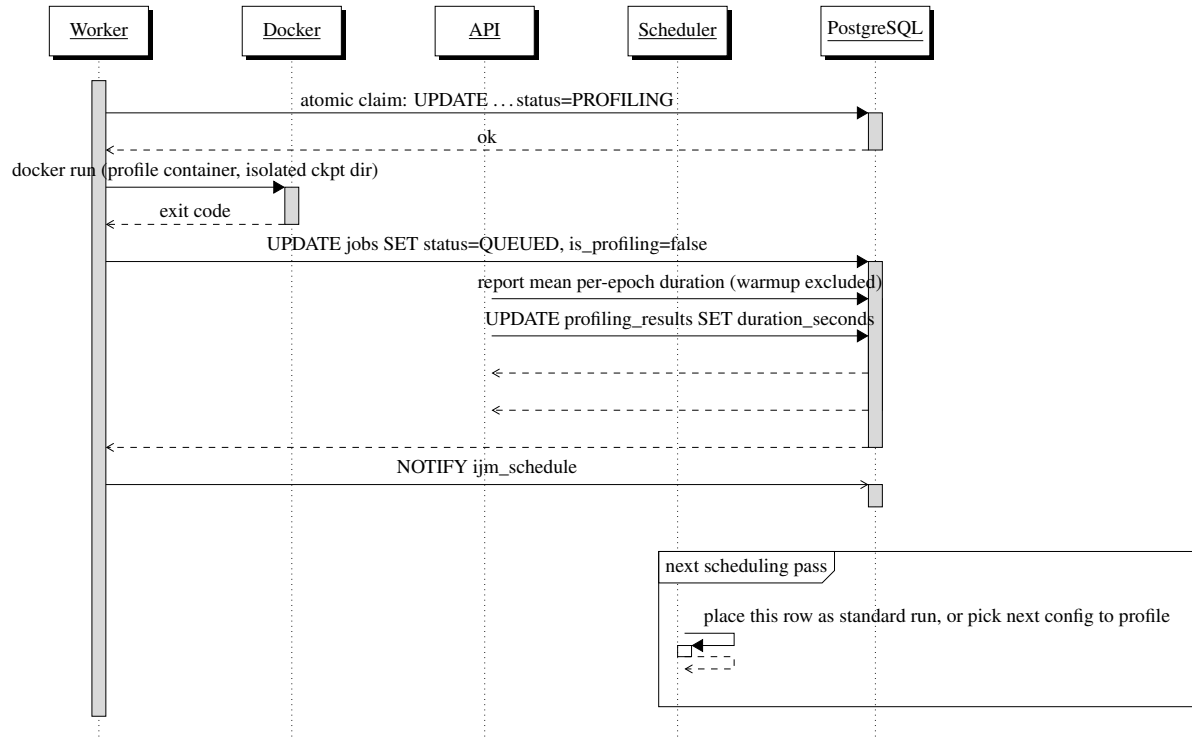


Figure 3: Profiling cycle. Profiling results are keyed by the (job-type, gpu-config) pair and shared across submissions of the same type. Each instance contributes one profile run by default (PROFILING_CONFIGS_PER_JOB); the per-instance claim row guards against duplicate measurements. Mean per-epoch time is computed from monotonic timestamps with the first (warmup) epoch excluded.

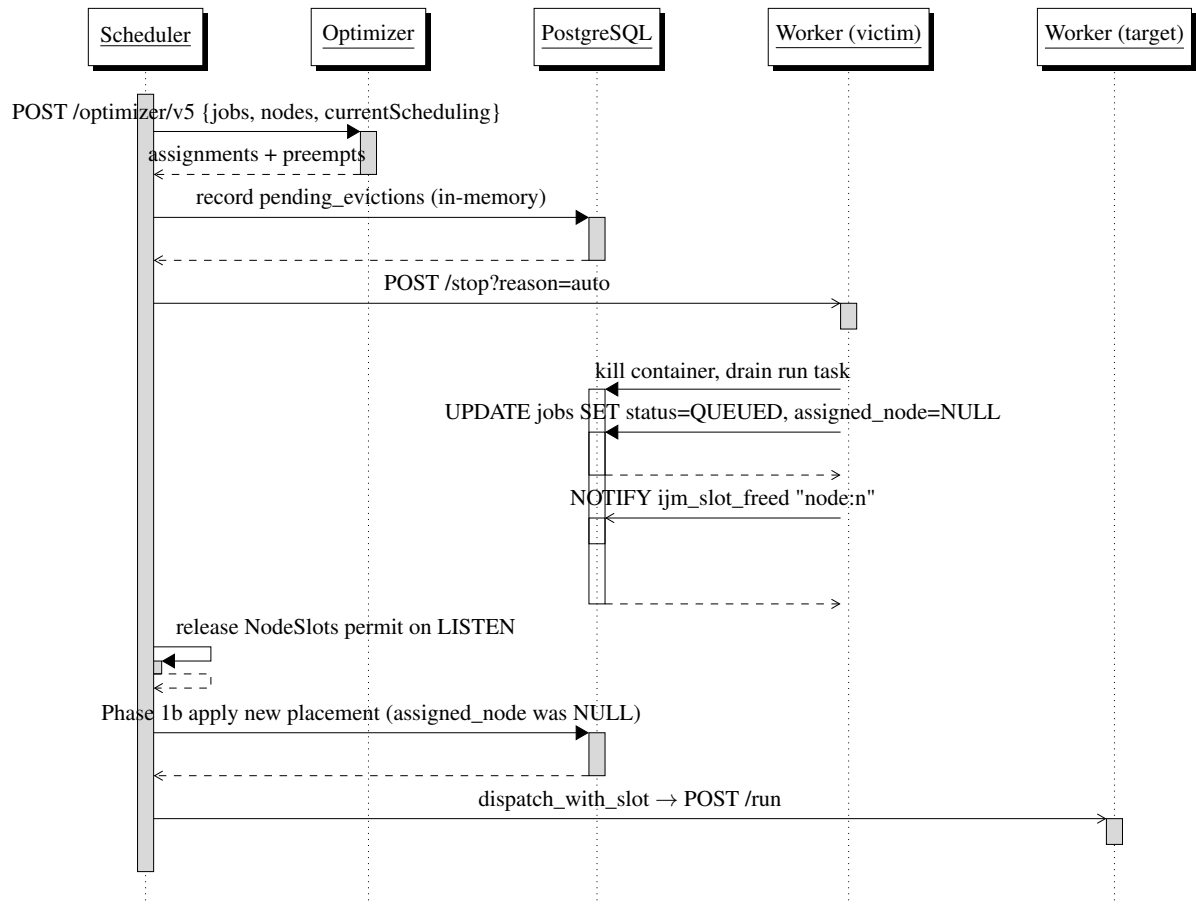


Figure 4: Optimizer-driven preempt and migrate. The API trusts GPUspb’s plan: any job in `currentScheduling` that the optimizer drops or moves to a different slot is preempted with `reason=auto`, which clears its assignment so Phase 1b can re-apply the new placement. An in-memory pending-evictions map makes freshly-evicted slots visible to the scheduler before the worker’s slot-freed commit lands, so multi-victim profile preempts do not deadlock.

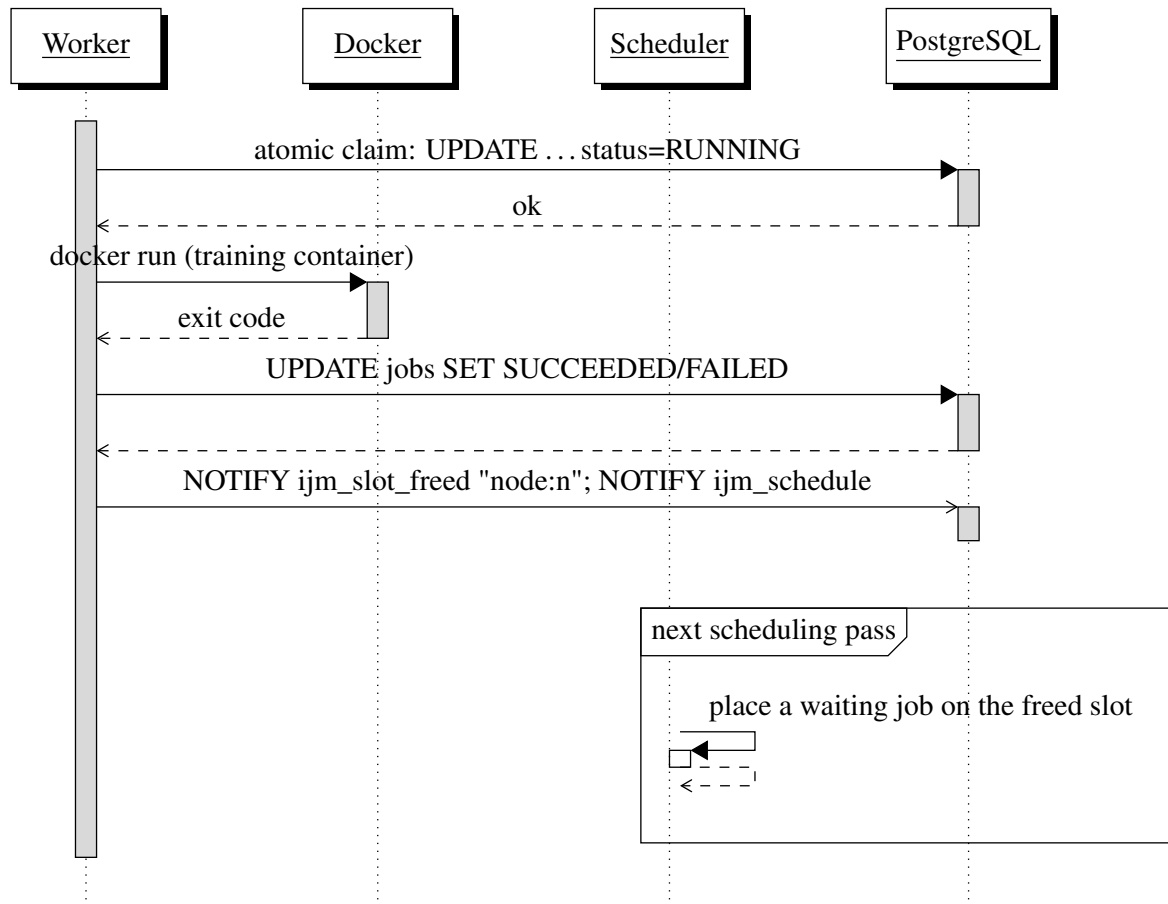


Figure 5: Standard run execution. The worker performs an atomic conditional `UPDATE ...WHERE status = ANY(RUNNABLE_STATUSES)` so concurrent stops / duplicate dispatches cannot race a row into `RUNNING`. On exit the worker fires both `ijm_slot_freed` (which releases the API's `NodeSlots` permit) and `ijm_schedule` (which wakes the scheduler) in the same transaction.

2.3 State machine

A job moves through the following states. All transitions are driven by atomic conditional `UPDATES` so concurrent stop / dispatch / completion paths cannot clobber each other's work.

From	To	Trigger
QUEUED	PROFILING / RUNNING	Worker Phase 1 atomic claim
PROFILING	QUEUED	Profile run finished; <code>is_profiling_run</code> flips to false
RUNNING	SUCCEEDED / FAILED	Container exit
RUNNING	PREEMPTED	User <code>POST /jobs/{id}/stop?reason=user</code>
RUNNING	QUEUED	Auto-preempt <code>reason=auto</code> : clears <code>assigned_node</code> so the next pass can place elsewhere
PREEMPTED / FAILED	QUEUED	<code>POST /jobs/{id}/resume</code>

Two distinct stop reasons separate *user-initiated* preemption (sticky `PREEMPTED`, the user must explicitly resume) from *scheduler-initiated* preemption (`QUEUED` with cleared assignment, picked up automatically on the next scheduling pass). The worker enforces this distinction and is also responsible for ignoring stale `/stop` calls aimed at a row that has since been re-assigned to a different node.

2.4 Concurrency invariants

The control plane relies on three invariants, each defended in code:

- **No GPU oversubscription.** Every dispatch goes through `NodeSlots.acquire(node_id, n_gpus)`. On startup the semaphores are reconciled from the DB (rows in `RUNNING/PROFILING` or `QUEUED` with an assigned node pre-acquire their permits). A periodic drift watcher recomputes the authoritative count every five minutes and corrects any divergence.
- **No duplicate containers per row.** Both the worker (Phase 1 claim, `running_jobs[job_id]` check) and the API (`dispatch_tasks[instance_id]` deduplication) refuse a second concurrent dispatch for the same instance. The reaper that resets stuck `QUEUED` rows cancels any live dispatch task instead of double-releasing its permit.
- **Profiling is sacred.** `Auto-preempt (_preempt_and_release)` refuses to act on `is_profiling_run=TRUE`; `profile-preempt (Phase 1c)` filters victims by `status='RUNNING'` only. Only an explicit user `/stop?reason=user` can cancel an in-flight measurement.

2.5 Cross-node checkpoints and image variants

Two GPU generations are supported. The `matemagician` node is pinned to driver 418.x (CUDA 10.1 max), so it runs a separate `:legacy` image built on PyTorch 1.5.1 + Python 3.7. Newer nodes run the default `:latest` image on PyTorch 2.6 + CUDA 12.4. Checkpoints are saved in the legacy pre-1.6 torch serialization format on every node so that a job can resume across nodes regardless of which torch version produced the file.

The shared `data/` directory is mounted on every node via `rclone` over SFTP and contains:

- `data/checkpoints/{job_id}/` — per-job checkpoints, mounted into the container at `/checkpoints`;
- `data/runs/{job_id}/` — captured stdout (`output.log`) and TensorBoard artifacts, mounted at `/runs`;
- `data/shared/data/` — read-only dataset cache (MNIST, CIFAR-10), mounted at `/runs/data`.

Worker-side environment variables `WORKER_GPU_MODE` (`runtime` | `cdi` | `none`) and `IMAGE_TAG_OVERRIDE` let the same `nodes_config` drive heterogeneous nodes: the `polimi-gpu` rootless daemon uses CDI device injection, `matemagician` rewrites `:latest` to `:legacy`, and a CPU-only sandbox can be added by setting `WORKER_GPU_MODE=none`.

3 Configuration & Job Submission

This section describes the system configuration files and the job submission API.

3.1 Cluster Configuration

Cluster topology is defined in two JSON files loaded at startup (paths configurable via environment variables).

3.1.1 Node Configuration (nodes_config.json)

Each entry describes a compute node with its GPU resources.

```
[
  {
    "id": "matemagician",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8001",
    "cost": 0.06,
    "resources": [
      { "gpu_type": "QuadroP600", "gpu_count": 2 }
    ]
  },
  {
    "id": "polimi-gpu",
    "isForProfiling": true,
    "workerUrl": "http://localhost:8002",
    "cost": 0.55,
    "resources": [
      { "gpu_type": "A40", "gpu_count": 2 }
    ]
  }
]
```

Field	Type	Description
id	string	Unique node identifier
isForProfiling	bool	If true, the node can host profiling runs. Production jobs may still land on it; profiling jobs may only land on it
workerUrl	string null	HTTP base URL of the worker on this node, typically reached through an SSH tunnel. When null, dispatch falls through to the in-process JobRunner (only useful for the docker-compose all-in-one development setup)
cost	float	Idle cost of the node (\$/h) used by the optimizer
resources	array	GPU resources attached to the node
resources[].gpu_type	string	GPU model name (e.g. A40, L40S, QuadroP600)
resources[].gpu_count	int	Number of GPUs of this type

Table 1: Node configuration fields.

GPU type and count are declared explicitly in the configuration rather than auto-detected at runtime. This allows mixed-GPU nodes (e.g. a node with both A40 and L40S GPUs), which ANDREAS did not support, and lets the operator declare nodes that aren't physically attached to the API host — the API only ever talks to workers over HTTP.

3.1.2 GPU Energy Costs (gpu_energy_costs.json)

Maps GPU type and count to hourly energy cost (EUR/h), used by the scheduler.

```
{
  "A40": { "1": 0.15, "2": 0.28, "4": 0.54 },
  "L40S": { "1": 0.18, "2": 0.34, "4": 0.65 }
}
```

3.2 Job Submission

Endpoint: POST /jobs

3.2.1 Request Body

```
{
  "job_id": "LSTM-small",
  "dockerImage": "ijm-lstm-small:dev",
  "scriptPath": "train.py",
  "directoryToMount": "/data/my-project",
  "Priority": 3,
  "deadline": "2026-03-20T18:00:00",
  "batchSize": 64,
  "profilingEpochsNo": 3,
  "epochsTotal": 20
}
```

Field	Type	Required	Description
job_id	string	yes	Reusable job type identifier; profiling results are correlated across submissions sharing the same job_id
dockerImage	string	yes	Docker image to execute
scriptPath	string	no	Path to training script (overrides Docker CMD with python3 <scriptPath>)
directoryToMount	string	no	Host directory mounted into the container at /workspace
command	string[]	no	Command and arguments (defaults to image entry-point; overridden by scriptPath)
Priority	int 1–5	no	Job priority (default 3)
deadline	datetime	no	Job deadline (ISO 8601)
batchSize	int	no	Batch size passed as BATCH_SIZE env var
profilingEpochsNo	int ≥ 1	no	Epochs for profiling runs (default 3)
epochsTotal	int ≥ 1	no	Total training epochs (default 20)

Table 2: Job creation request fields.

Each submission receives a unique UUID instance ID (id) for lifecycle tracking, separate from the job_id type identifier. When scriptPath is provided, the container runs python3 <scriptPath>; otherwise the Docker image’s default CMD is used. Unlike ANDREAS, Priority and deadline are optional, and an additional command field provides flexibility for non-standard workloads.

3.2.2 Response (201 Created)

Returns the full job object including scheduler decisions:

```
{
  "id": "a1b2c3d4-...",
  "job_id": "LSTM-small",
  "image": "ijm-lstm-small:dev",
  "command": ["python3", "train.py"],
  "status": "QUEUED",
  "is_profiling_run": true,
  "assigned_node": "node-mixed-prof",
  "assigned_gpu_config": { "A40": 1 },
  "priority": 3,
  "epochs_total": 20,
  "profiling_epochs_no": 3,
  "created_at": "2026-03-18T10:00:00Z",
  "updated_at": "2026-03-18T10:00:00Z",
  ...
}
```

The response immediately shows the scheduling decision: assigned node, GPU configuration, and whether this is a profiling run.

3.3 Full API Overview

Table 3 lists all endpoints. Interactive documentation is available at `/docs` (Swagger UI) and `/redoc` when the API is running.

Method	Path	Description
GET	/	Health check (basic)
GET	/health	Detailed health (DB + job runner)
GET	/nodes	List cluster nodes with status
GET	/gpu-costs	GPU energy cost table
GET	/configurations	Valid GPU configurations
POST	/jobs	Submit a new job
GET	/jobs	List all jobs (paginated)
GET	/jobs/{id}	Get job by ID
DELETE	/jobs/{id}	Delete a job
DELETE	/jobs	Delete all jobs
POST	/jobs/{id}/stop	Stop a running job
POST	/jobs/{id}/resume	Resume a stopped/failed job
GET	/jobs/{id}/logs	Retrieve job output log
GET	/profiling-results/{id}	Profiling results for a job type

Table 3: API endpoints.

All scheduling, profiling, and dispatch decisions are consolidated into a single FastAPI service, in contrast to ANDREAS which split these across a Java Jobs Manager and a C++ Jobs Optimizer. The actual execution lives in per-node *workers* (also FastAPI), which the API reaches over HTTP through SSH tunnels. An optional external GPUspb optimizer is reached the same way: when its URL is unset the system falls back to the in-API greedy scheduler. IJM exposes explicit `stop`, `resume`, `logs`, and

delete endpoints, as well as `/profiling-results` and `/configurations` to give users visibility into the scheduler’s state.

3.4 Runtime environment

The most important runtime knobs (all set in the API process unless noted otherwise) are:

Variable	Default	Description
DATABASE_URL	postgresql://.../ijm	PostgreSQL DSN, reached via SSH tunnel
OPTIMIZER_URL	unset	When set, the API delegates placement to the GPUspb HTTP service. Unset $\Rightarrow$ greedy scheduler only
OPTIMIZER_METHOD	RG	GPUspb solver method passed in the request body
OPTIMIZER_VERBOSE	0	GPUspb per-call log verbosity (0–3)
OPTIMIZER_SWITCH_PENALTY_S	60	Wall-time penalty (s) charged when judging optimizer migrations without destination profiling data
IJM_OPTIMIZER_GUARD_MIGRATIONS	unset	When set, re-enables the local cost-benefit filter on optimizer-proposed migrations (default trusts GPUspb)
PROFILING_CONFIGS_PER_JOB	1	How many distinct GPU configs a single job <i>instance</i> profiles before falling through to a standard run
EXECUTOR	docker	Executor backend for the in-process JobRunner: <code>docker</code> or <code>mock-slurm</code> (workers always use the docker CLI)
WORKER_GPU_MODE	runtime	Worker-side; how Docker exposes GPUs to training containers: <code>runtime</code> / <code>cdi</code> (rootless) / <code>none</code>
IMAGE_TAG_OVERRIDE	unset	Worker-side; rewrites <code>:latest</code> to the given tag. Used on <code>matemagician</code> to pick the <code>CUDA-10.1-compatible</code> <code>:legacy</code> image
HOST_ROOT, HOST_PROJECT_ROOT	/host, \${PWD}/..	Resolve container-internal vs. host paths for Docker bind mounts

Table 4: Key environment variables.

4 End-to-End Scenarios

4.1 Objective and cost model

The scheduler minimises

$$\text{TOTALCOST} = \sum_j \text{GPUcost}(j) + \sum_j \text{TardWeight}(j) \cdot \max(0, \text{finish}(j) - \text{deadline}(j)),$$

where GPUcost is wall-clock time times the bundle hourly rate (Table 5, left) and TardWeight grows with job priority (Table 5, right). With slack deadlines the cheapest plan wins; under deadline pressure the tardiness term shifts urgent jobs onto the fastest available bundle.

GPU bundle	\$/hour	Priority	TardWeight (\$/sec)
QuadroP600×1	0.030	1	0.000254
QuadroP600×2	0.055	4	0.000396
A40×1	0.300	5	0.000444
A40×2	0.550		

Table 5: Bundle hourly rates and per-priority tardiness weights. Two-GPU bundles include a small co-location discount.

Cluster. Two profiling-capable nodes, four slots total:

Node	GPUs	Bundle hourly cost
matemagician(n1)	2×QuadroP600	0.055
polimi-gpu(n2)	2×A40	0.550

For each new job type the profile sweep exercises all four (node, GPU-count) cells (configs_per_job=2, four instances).

4.2 Two-GPU placement

The `cnn_big` workload (ten 3×3 conv blocks, 3→64→128→256→384→512 channels, 128×128×3 inputs, batch 32) is sized so per-step compute amortises DataParallel sync overhead on both GPU classes. Table 6 reports per-epoch wall-clock times from the profile sweep (`SelectJobs_times.csv`).

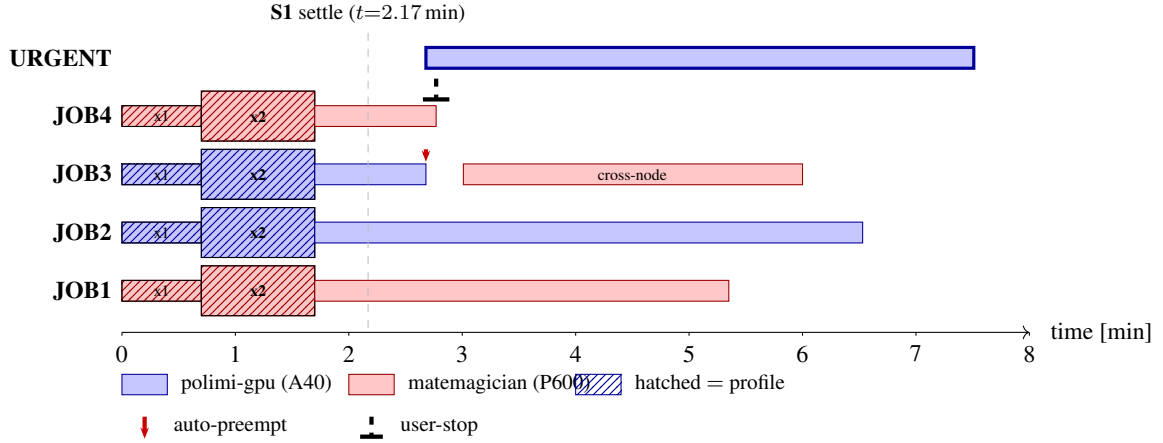
Bundle	per-epoch [s]	rel. 1-GPU	GPUcost/epoch [\$]
A40×1	6.294	–	0.000525
A40×2	5.626	1.12× faster	0.000859
QuadroP600×1	51.064	–	0.000426
QuadroP600×2	30.322	1.68× faster	0.000463

Table 6: Measured per-epoch wall-clock and GPUcost for `cnn_big`; both 2-GPU bundles beat their 1-GPU sibling.

Over 40 epochs the four plans cost \$0.0210, \$0.0344, \$0.0170, \$0.0185 (A40×1, A40×2, P600×1, P600×2 respectively). P600×1 is the cost-minimum for slack deadlines. P600×2 is only \$0.0015 more expensive but 41% faster, so any deadline that P600×1 misses by more than 3.4 s at priority 5 flips the ranking to P600×2; this is the cell exercised in Scenario 2 Stage 3.

Label	Type	Priority	Deadline	Submitted at
JOB1	lstm-small	4	+1 h	$t = 0$ s
JOB2	lstm-small	4	+1 h	$t = 5$ s
JOB3	lstm-small	4	+1 h	$t = 10$ s
JOB4	lstm-small	4	+1 h	$t = 15$ s
URGENT*	lstm-small	5	+90 s	end of S1

Table 7: Scenario 1 submissions.

Figure 6: Scenario 1 timeline (v9, 2026-05-12). Taller bars in the profile phase are the 2-GPU profile cells. Slot count ≤ 2 on each node at every instant.

4.3 Scenario 1 — single type

Four patient `lstm-small` jobs plus one URGENT (Table 7). Table 8 gives the per-bundle ExecTime and GPUcost the optimiser saw at the S1 round.

Bundle	per-epoch [s]	GPUcost/epoch [\$]	40-ep cost [\$]	Rank
A40×1	7.25	0.000604	0.0242	—
A40×2	9.12	0.001393	0.0557	worst
QuadroP600×1	5.47	0.0000456	0.00182	best
QuadroP600×2	10.66	0.000163	0.00651	—

Table 8: Per-bundle cost the optimiser sees at the S1 round; slack deadlines so TardCost= 0.

Settled placement: JOB1, JOB2 → `polimi-gpu A40×1` (overflow); JOB3, JOB4 → `matemagician P600×1`. At S2 the URGENT comparison (Table 9) chooses A40×1, evicting JOB2.

Plan for URGENT	ExecTime [s]	GPUcost [\$]	Tardiness [s]	TardCost [\$]	TotalCost [\$]	Decision
A40×1 (preempt JOB2)	217	0.0181	127	0.0564	0.0745	✓ chosen
P600×1 (no preempt)	442	0.0037	352	0.1563	0.1600	rejected

Table 9: Optimiser plan comparison for URGENT at the S2 round.

Measured run (v9). S1 settles at $t=2.17$ min with all four profile cells exercised ($\{A40\times1, A40\times2, P600\times1, P600\times2\}$). URGENT (7d7c848a) placed in 6 s; auto-preempt victim JOB3 (0f6f54c4 on `polimi-gpu`). Operator user-stops JOB4 (c957097f; sticky PREEMPTED). JOB3 resumes from

checkpoint within the 10-minute Stage 4 budget. Terminal at $t=14.07$ min: 4 SUCCEEDED, 1 PREEMPTED, 0 FAILED. Slot tracker: acquire_oversub_count= 0, release_underflow_count= 1, drift_recovery_count= 60.

4.4 Scenario 2 — two types

Label	Type	Priority	Deadline
JOB1	S (lstm-small)	1	+8 h
JOB2	B (cnn_big)	1	+8 h
JOB3	S (lstm-small)	4	+1 h
JOB4	B (cnn_big)	4	+2 h
URG-S* (S2)	S	5	+90 s
URG-B* (S3)	B	5	+90 s
JOB7 (S4.5)	opposite of user-stop victim	1	+8 h

Table 10: Scenario 2 submissions.

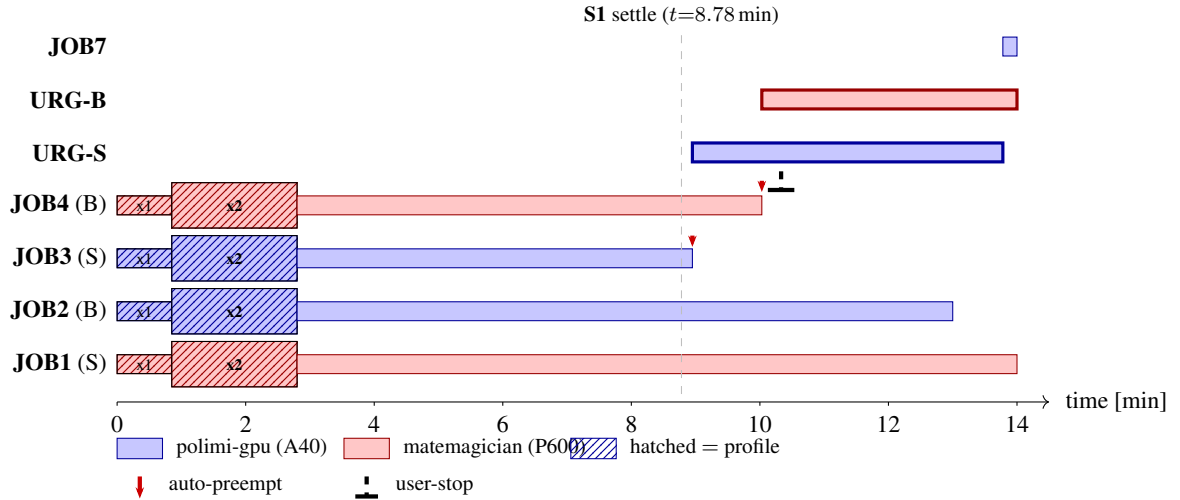


Figure 7: Scenario 2 timeline (v14, 2026-05-13). Taller bars are the 2-GPU profile cells. URG-S pre-empts JOB3 at $t=8.95$ min; URG-B pre-empts JOB4 at $t=10.03$ min; user-stop on JOB4 at $t=10.33$ min; JOB7 queued at $t=10.38$ (dotted) and dispatched once URG-S vacates A40 at $t=13.78$.

Submissions in Table 10: four patient jobs split across two types, two back-to-back URGENT submissions at S2/S3, and a late JOB7 at S4.5.

Type	Bundle	per-epoch [s]	40-epoch [s]	rate [\$ /h]	40-ep cost [\$]
S	A40×1	7.25	290	0.300	0.0242
S	A40×2	9.12	365	0.550	0.0557
S	QuadroP600×1	5.47	219	0.030	0.00182
S	QuadroP600×2	10.66	426	0.055	0.00651
B	A40×1	6.29	252	0.300	0.0210
B	A40×2	5.63	225	0.550	0.0344
B	QuadroP600×1	51.06	2042	0.030	0.0170
B	QuadroP600×2	30.32	1213	0.055	0.0185

Table 11: Per-bundle profile data the optimiser sees at the S1 round.

Settled S1 placement. JOB1, JOB3 (lstm-small) $\rightarrow$ matemagician P600 $\times$ 1; JOB2, JOB4 (cnn_big) $\rightarrow$ polimi-gpu A40 $\times$ 1. The overflow ranking is by “cost lost if forced off the cheap node”: lstm-small loses \$0.0224, cnn_big loses \$0.0040, so lstm-small keeps both P600 slots. Total \$0.0456.

URGENT	Plan	ExecTime [s]	GPUcost [\$]	Tard [s]	TardCost [\$]	Total [\$]	Decision
URG-S	QuadroP600 $\times$ 1 (preempt JOB1)	219	0.00182	109	0.0484	0.0502	✓ chosen
URG-S	A40 $\times$ 1 (preempt JOB2)	290	0.0242	180	0.0799	0.1041	rejected
URG-B	A40 $\times$ 1 (preempt JOB2)	252	0.0210	162	0.0719	0.0929	✓ chosen
URG-B	A40 $\times$ 2 (preempt JOB2+JOB4)	225	0.0344	135	0.0599	0.0943	2nd (Δ 1.5%)
URG-B	QuadroP600 $\times$ 2 (preempt JOB1+JOB3)	1213	0.0185	1123	0.4986	0.517	3rd
URG-B	QuadroP600 $\times$ 1 (preempt JOB1 or JOB3)	2042	0.0170	1952	0.8667	0.884	4th

Table 12: Optimiser plan comparison for URG-S (S2) and URG-B (S3).

URGENT plans at S2 and S3.

Measured run (v14). S1 settles at $t=8.78$ min with profile coverage $4\times 2 = 8$ cells. URG-S (8d875cc9) placed in 17 s; auto-preempt victim JOB3 (d47acbf3, lstm-small on polimi-gpu). URG-B (5e9b9820) placed in 24 s; auto-preempt victim JOB4 (9658d9df, cnn_big on matemagician). Stage 4 verifies ≥ 1 cross-node resume: JOB3 receives assigned_node=matemagician (cross-node from its polimi-gpu origin); JOB4 resumes on its origin matemagician. Operator user-stops JOB4 at $t=10.33$ min; late JOB7 (fd074cb2) submitted at $t=10.38$ min and backfills polimi-gpu A40 once URG-S vacates ($t=13.78$ min).

4.5 Reproducibility

Every numerical entry in Tables 6, 8, 9, 11 and 12 is taken verbatim from the optimiser’s per-round inputs (Lof_Selectjobs.csv, SelectJobs_times.csv, GPU-costs.csv) and chosen plan (results/RG_schedule_*.csv) under <optimizer>/calls/call_<ts>.0/. The two scenarios are driven by infra/e2e_scenario.sh and infra/e2e_scenario_2types.sh, both of which print one line per assertion and expose the slot tracker via GET /admin/slots.

References

- [1] Danilo Ardagna, Federica Filippini, et al. ANDREAS – artificial intelligence training scheduler for accelerated clusters. Technical report, Politecnico di Milano / TETRAMAX, 2021. Horizon 2020 project deliverables D1–D3.